

RevView/OMNI

Review Helper Application Concept

1. Overview

RevView/OMNI is a web application designed to centralize and automate parts of the formal review process used for safety-critical avionics software. The goal is to remove the repetitive coordination and data-entry work surrounding reviews so engineers can spend more time focusing on the actual review.

The current process is spread across Jira, SVN, Jenkins, review checklists, review records, diff files, source code, and other files. The application aims to connect the files into one controlled workspace while preserving the review artifacts and human judgement required by the existing process.

The application will ultimately be useful as both an internal engineering tool and as a senior design project for TCU students. Students will gain experience working with avionics systems and software engineers to design and implement a system around an actual formal code review process.

2. Background and Current Workflow

The current code review process is slow, manual, and fragmented across multiple tools. Authors must gather review checklists, create diff files, fill out review records, collect SVN information, perform builds when needed, and notify reviewers. Reviewers then work across several open resources, including the diff, source code, Jira ticket, spreadsheet checklists, and related files.

This creates unnecessary administrative work, increases the chance of making mistakes, and slows down a process that the team performs frequently.

2.1 Author-side work

For each review, an author may need to:

- Gather up to three review spreadsheet checklists for the platforms affected.
- Fill out review descriptions for each changed file across code/tools/tests.
- Locate and enter the SVN revision number and the repository link for each file.
- Fill out the review record.
- Create the diff file and ensure all changed files are included.
- Perform a Jenkins test build.
- Notify reviewers that the review has started.

2.2 Reviewer-side work

Reviewers may need to keep several resources open at the same time:

- The diff file.
- The source code.
- The Jira ticket.
- The Excel checklist.
- Any related files needed for context.

Reviewer comments are also inconvenient to document because they often require cross-referencing files, line numbers, checklist items, and Jira ticket details. This increases cognitive load and makes it easier for important details to be missed.

2.3 Recurring problems

Manual data entry is time-consuming. Authors spend significant time filling out information that already exists in Jira, SVN, and Jenkins.

Important review sections can be missed. A code review can have up to nine parts. If a review has multiple code/tools/test sections across the three platforms, reviewers may accidentally miss completing some parts.

Version mismatches can happen. Reviewers may forget to update their working copy and comment on outdated code.

Review context depends heavily on the author. If the author does not clearly describe the change, reviewers may struggle to understand the purpose and risk of each file change.

Reviewers may miss that they have been added to a review. Reviewers are notified by email when they have been added to a review, which can easily get lost in an inbox.

Review comments are disconnected from the code. Comments often require references such as “line 42: fix this,” which is harder to manage than comments that are directly attached to the relevant code location.

The spreadsheet-based process does not scale well. Multiple spreadsheets, review records, and platform-specific checklists make the process harder to maintain and audit.

3. Product Goal

The goal of the application is to turn a fragmented review process into one connected workflow. An author will be able to start a review by selecting a Jira ticket, verify the information the application gathers, select reviewers, and send the review without manually assembling the review package across multiple tools.

The application will handle repetitive setup work in the background while allowing the author to verify and adjust generated information before the review is sent out.

3.1 Core author workflow

1. Select the Jira ticket.
2. Review the automatically populated information.
3. Add or edit descriptions where needed.
4. Select reviewers.
5. Start the review.

4. Automation

Automation is the foundation of the application. Without automation, the tool can still be used to organize review materials, but it would not fully solve the current workflow problems.

4.1 Core automation

- Start a review by selecting a Jira ticket.
- Find the matching SVN branch or branches based on the Jira ticket name.
- Create diff files automatically for the relevant platforms and code/tools/tests categories.
- Populate review fields, including file name, SVN URL, revision number, and last author.
- *Populate review-time information such as reviewer and review date.**
- Start a new Jenkins build when applicable.
- Link the test build result directly to the review.
- Consolidate review checklists and update the review record when requested by the moderator.

**This information may be automated. More information in Section 6.*

4.2 Notifications and workflow updates

- Notify reviewers when a review starts.
- Notify moderators and authors when a checklist is submitted or updated.
- Notify moderators when all reviews' checklists are submitted.
- Notify all relevant users when the review is closed.
- Move the Jira ticket to "In Close Out" when the moderator closes the review.

5. AI-Assisted Features

AI should be implemented as a supporting feature, not as the purpose of the application. The project should aim to minimize AI bloat and ensure that AI does not replace reviewer judgment.

For example, when generating a “reason for file change,” AI should not invent the reason the author made a change. Instead, it can summarize the objective changes shown in the diff. The actual reason for the engineering decision belongs to the author.

5.1 Change description

AI could draft descriptions for each changed file using context from:

- The diff file.
- Related Jira ticket details.
- Requirements.
- Related files or dependencies.
- Commit messages.

The author reviews, edits, and approves any generated description before the review is sent.

5.2 Minor issue detection

AI or deterministic automated rules could identify low-value issues such as:

- Blank lines.
- Inconsistent indentation.
- Outdated copyright headers.
- Formatting inconsistencies.
- Simple style violations.

The purpose is to let authors fix low-value issues before reviewers spend time on them.

5.3 Preliminary compliance checks

The application could perform a preliminary compliance check and suggest results such as:

- Compliant
- Not compliant
- Not applicable
- Needs reviewer judgement

For “Compliant,” “Not compliant,” or “Not applicable” suggestions, the application should provide a reason, such as:

- A missing required header.
- A code snippet that violates a checklist condition.
- A file type that does not require a particular criterion.

These suggestions would reduce the amount of manual checklist work, but reviewers would still approve or override the result.

5.4 AI involvement levels

Teams or organizations may control the level of AI involvement. A three-level model provides a way to keep AI proportional to the value it provides.

Level	Role	Outcome
Level 1: None	No AI. The application acts as an organization and workflow tool.	Centralize review materials, connect Jira, diffs, checklists, and review records, reduce manual coordination.
Level 2: Necessary	Minimal AI used only where it clearly reduces manual effort.	Draft file descriptions, recommend reviewers, clarify changes from diffs, summarize ticket information, flag obvious formatting or checklist issues.
Level 3: Helper	AI acts as an additional review assistant.	Identify areas of concern, suggest likely defects, highlight riskier changes, perform a preliminary review before humans begin.

Level 2 is likely the most practical starting point because it supports the review process without making AI intrusive. If AI identifies issues before the review is sent, the author may simply fix them first. In this case, AI is most useful for helping reviewers prioritize their attention rather than replacing the review. Level 3 may be useful for large diffs or complex reviews, but should be carefully scoped.

6. Automation Beyond AI

Not every intelligent feature needs generative AI. The application should prefer to use deterministic rules or traceable predictive models when they are more appropriate.

Checklist compliance should be converted into rule-based validation where possible.

A review-duration estimator is another possible data-science component. If the system stores historical review duration along with features such as the number of files and lines changed, a regression model could estimate how long a future review may take for a user. The same information could help inform and automatically recommend a due date.

Recommended reviewers could also be selected with a classification algorithm based on historical review information and reviewer availability.

This type of decision-making is preferred over generative AI when traceability and modifiability are important. These automation problems could be especially interesting for students with

mathematics and data science backgrounds. The project can enable students to explore the decision to automate tasks that might initially seem easier to solve with AI while keeping the resulting decisions explainable and controllable.

7. Review Package and Traceability

The application should provide an entirely new interface so users do not have to work directly in an Excel spreadsheet or a Word document during a review. However, the same review information must still be captured and exportable as the original filetype so that the existing review package can be produced for audits.

When a review is created, the application should generate and maintain the required artifacts:

- Review record.
- Code review checklist(s).
- SVN-generated diff file.
- Build and test results.
- Additional files supplied by the author.

Checklists and review records must still be auditable and conform to requirements. The application should generate and package all required artifacts.

7.1 Review record

Fields should be automatically populated wherever possible. The author must still provide information that requires human input, such as the *type of review, method, date, return date, reviewers, and other details*.*

**This information may be automated. More information in Section 6.*

7.2 Review checklist

Checklist fields should be populated automatically wherever the required information can be determined from the connected systems. Judgement-based fields should remain for the author to complete. A missing field such as “reason for file change” should prompt the user for input unless the AI assistance feature is enabled.

7.3 Diff

The required SVN diff should be generated automatically and verify that all changes match with the files in the checklist. The author does not necessarily need to review the entire diff before starting the review, particularly when many files have changed, but the application should prompt the author to verify it.

7.4 Version history and review rounds

- Track version history and review rounds more effectively.
- Capture the current revision and the revision at review creation so that new diffs can be generated easily after reviewer-requested changes have been made.
- Allow authors or other users to generate another diff whenever needed instead of avoiding it because of manual overhead.
- Link reviews to older reviews to improve traceability.
- Extend the system to merge reviews and requirement reviews as a possible stretch goal.

8. Technical and Process Considerations

- Diff files must be developed with SVN because they are required review artifacts. Branches with multiple updates and development merges make it difficult to isolate author-only modifications. This requires a technical solution since SVN does not handle it natively.
- Review information should be stored in Excel/CSV where required by the existing process or audit workflow. The application should maintain the existing review information and export it to the old review checklist even though users interact with a new interface.
- Adding a reviewer through Jira should notify them in the application.
- Adding a reviewer through the application should also add that reviewer to the Jira ticket.

9. MVP Scope

The first version should focus on automating manual steps and providing basic AI integration.

Create review from Jira ticket

- Select a ticket.
- Pull the ID, title, description, status, linked requirements, and related metadata.

SVN integration

- Find the matching branch.
- List changed files.
- Generate diff files.
- Capture revision number and repository links.

Unified review workspace

- Show source code, issue details, changed files, diff, checklist, comments, and build status in one place.

Automated checklist population

- Pre-fill fields that can be determined from integrations.
- Leave judgement-based fields for reviewers.

Basic AI assistance

- Draft file change descriptions.
- Summarize diffs.
- Flag simple formatting or compliance concerns.

Notifications

- Notify reviewers when a review starts.
- Notify the author when comments are added.
- Notify relevant users when the review is closed.

Review closure

- Consolidate final review results.
- Update the review record.
- Gather all files and generate a review package.
- Move the Jira ticket to “In Close Out.”

10. Success Metrics

The primary success metric is the reduction in time required to start a review after the author has already completed the required Jenkins build(s). The current estimate works under the premise that a review takes approximately 20 minutes to gather required forms and fill in fields under the existing process.

Scenario	Estimated user effort	Expected improvement
Current process	20 minutes to gather forms, fill fields, excluding Jenkins build time.	Baseline.
Application with automation features only	7 minutes for written work plus 2-3 minutes to review generated artifacts.	50-55% reduction in review-start time.
Application with automation and AI features	2-5 minutes to review generated fields and artifacts.	75-90% reduction in review-start time.

These estimates are based on the current concept and should be validated with actual users once a prototype is available.

11. Senior Design Fit and Student Value

The project will be presented as a large integrated application for safety-critical avionics systems. The project itself is not safety-critical.

To strengthen credibility, the project should emphasize real engineering constraints. Leveraging our work with large aerospace organizations will provide meaningful context on the scale and impact of the work.

The intention is not for the project to become another senior design project that is reintroduced to next year's class. The desired outcome is an application that can be deployed internally as soon as it meets the necessary criteria.

Students will work directly with avionics systems and software engineers to design the application. The sponsors are engineers and TCU alumni who understand the process and can communicate specific requirements rather than presenting a vague idea for students to figure out themselves.

Students would gain exposure to practices used in DO-178C development and learn how formal code reviews are conducted for safety-critical avionics software while also developing a system around that process.

The project's educational value comes from understanding why formal reviews are necessary in safety-critical systems. Students will learn about the role of software assurance, traceability, review artifacts, version control, testing, and importance of human judgement in a safety-critical development process.

The resulting system should be useful to our team and should support software processes associated with aircraft and spacecraft.

12. Constraints

IT cannot add a team of students as users on our internal network. This rules out direct student access to our production Jira, SVN, and Jenkins instances for development and testing. This network-access constraint will change the development environment that students will work with. The plan is to give students the same integration experience without adding any student accounts, devices, or network paths to our internal systems.

12.1 Sandboxed development environment

The sandbox should run actual Jira, SVN, and Jenkins instances, hosted with no route to our network, so students learn how the platforms behave, including authentication, API responses, and the branch and diff behavior described in Section 8. Because no student account, device, or network path touches our internal systems, this satisfies the IT constraint directly while still giving students the real integration experience the project depends on.

Populating the sandbox with mock data that mirrors our workflow structure lets students explore realistic ticket flows, branching, diffs, and builds, so the integration code they write transfers cleanly once the software team points it at our production.

12.3 Environment components

- **Jira:** Atlassian's free tier supports up to ten users at no cost, which covers the student team and sponsor accounts, hosted entirely on Atlassian's infrastructure with no connection to our production Jira.
- **SVN:** Apache Subversion is open source and inexpensive to self-host on a small cloud or university-provided VM, isolated from our network.
- **Jenkins:** Also open source and self-hostable on the same VM or a second small instance.

12.2 Mock data

The sandbox should be populated with synthetic data that mirrors our real workflow rather than any actual proprietary content. This includes:

- Jira boards and ticket types that match our review workflow.
- A checklist with the same structure as our real checklists, populated with placeholder data.
- An SVN repository with branch and revision history representative of a typical review cycle.
- Sample diffs and build results that resemble what students would see in production.

12.3 Transition to production

- Confirm the sandbox tool versions and API behavior are reasonably close to our internal instances before development starts.
- Software team reviews the integration layer, auth, error handling, field mappings, etc. once students hand off the project.
- Swap sandbox configuration for production endpoints and credentials behind our network, then run the integration layer against a small set of real tickets before full rollout.

13. Iterative Integration Testing

Adhering to the Agile methodology to test features incrementally throughout development will allow us to catch problems as they arise. This will address two problems:

- Weekly student demonstrations of the application will work against the sandbox, but it does not confirm that the feature works against our production Jira, SVN, and Jenkins instances.
- Waiting until handoff to check the application against our actual system puts all of the integration risk at a single point at the end of each semester.

The student team will hand over work to the software team to test each completed component against our production systems on the earlier of:

- The last day of each sprint, regardless of what is finished, or
- Whenever the student team marks a feature complete

This keeps the software team in the loop about student progress, ensuring that any fixes can be addressed in the following sprint.

13.1 Verification process

- Student team tags a build with a demonstration and/or README once a feature is complete or when a sprint checkpoint is reached, whichever comes first.
- Software team verifies the integrity of the build and runs it against a test area within our production Jira, SVN, and Jenkins, such as a designated test board, repository, or job.
- Any discrepancies are logged by the software team and handed back to the student team to be addressed in the next sprint.